

Deploying an AI Platform on Kubernetes

After studying inference performance, the next question is how to deploy AI services in a way that can be routed, scaled, secured, monitored, and operated as a platform.

A performance lab can show where model-serving bottlenecks appear. But once an AI system grows beyond a local service, the platform needs a disciplined operating model. APIs, workers, model-serving services, databases, queues, observability tools, dashboards, secrets, and routing rules need clear boundaries.

That is the purpose of the `07-kubernetes-ai-platform-deployment` project.

This project uses Kubernetes to bring the major pieces of an AI application stack into a coherent deployment model. It defines service boundaries, routing paths, configuration management, resource limits, health checks, autoscaling rules, network policies, and production-readiness considerations.

From an AI Data Platform Architect's perspective, this project is less about deploying one more endpoint and more about defining the operational contract around the AI platform.

Kubernetes becomes the control plane for that contract.

Why Kubernetes Matters for AI Platforms

Modern AI systems rarely fail because one model call does not work.

They fail because the surrounding platform is not ready for real operating conditions.

Services cannot scale independently.

Traffic paths are unclear.

Secrets are handled casually.

Background jobs are coupled too tightly to user-facing APIs.

Model-serving workloads compete with lighter application workloads.

Stateful dependencies are treated as afterthoughts.

Observability arrives after the first incident instead of before it.

Kubernetes helps address these issues by giving the platform a way to define, deploy, scale, and manage each part of the system explicitly.

AI applications still have familiar platform needs:

- APIs
- databases
- queues
- services
- environment variables
- health checks
- ingress
- dashboards
- logs and metrics

But AI workloads add pressure in specific places.

RAG systems need vector storage, document metadata, and controlled retrieval paths.

LLM services need predictable routing, resource awareness, and concurrency boundaries.

Workers need to process asynchronous jobs without blocking the API tier.

Observability services need access to traces, metrics, retrieval behavior, token usage, and quality signals.

Databases and queues need to be modeled as platform dependencies, not hidden implementation details.

Kubernetes gives each of these concerns a deployment boundary.

That is the shift from an AI application to an AI platform.

Project Overview

The `07-kubernetes-ai-platform-deployment` project organizes the platform inside a dedicated Kubernetes namespace:

```
ai-platform
```

That namespace acts as the logical boundary for the stack. It groups the platform resources together and makes the system easier to inspect, deploy, or remove as a unit.

At a high level, the architecture looks like this:

```
User / Client
-> nginx Ingress at ai-platform.local
    -> API service
```

- > LLM service
- > Observability service
- > Dashboard service

API service

- > LLM service
- > Postgres / pgvector
- > Redis queue
- > Observability service

Worker service

- > Redis queue
- > Postgres

The major platform components are:

Component	Role
API service	Handles user-facing requests
Worker service	Processes asynchronous background jobs
LLM service	Represents model-serving capacity
Postgres	Provides durable state and vector-backed data patterns
Redis	Provides queueing and async coordination
Observability service	Exposes the reliability view of the platform
Dashboard service	Provides a user-facing or operator-facing dashboard surface
Ingress	Defines the public routing boundary
ConfigMaps	Externalize non-sensitive configuration
Secrets	Externalize sensitive configuration
HPAs	Define autoscaling behavior
NetworkPolicies	Start defining allowed traffic paths

This structure mirrors a production-style AI platform.

The API tier is the entry point.

The worker tier handles background work.

The model-serving tier is isolated behind its own service.

The data tier owns persistence.

The queue tier coordinates asynchronous processing.

The observability tier makes platform behavior visible.

The deployment layer defines how these pieces run together.

The Platform Architecture

The project's Kubernetes architecture is intentionally clear.

Each major concern maps to a Kubernetes object or set of objects.

Routing uses Ingress and Services.

Runtime isolation uses Deployments and Pods.

Configuration uses ConfigMaps.

Sensitive values use Secrets.

Persistence uses a PersistentVolumeClaim.

Scaling uses HorizontalPodAutoscalers.

Safety starts with resource requests, limits, readiness probes, liveness probes, and network policies.

This is the right architecture habit.

A platform should not rely on tribal knowledge to explain how services communicate. The deployment manifests should show the operating model.

The architecture answers practical questions:

What is public?

What is internal?

Which services receive traffic?

Which services are background-only?

Where does persistent data live?

How do workloads find Redis and Postgres?

How does the API reach the LLM service?

How are configuration values injected?

How are secrets separated from code?

How does Kubernetes know a pod is healthy?

How can services scale independently?

These are the questions that move the project from local application development into platform deployment.

Deployments as Service Contracts

In Kubernetes, a Deployment is more than a way to run a container.

It is a service contract.

The API service manifest demonstrates this idea. It runs multiple replicas, uses a rolling update strategy, defines CPU and memory requests and limits, loads configuration from ConfigMaps and Secrets, and exposes readiness and liveness probes.

That matters because user-facing services should not rely on hope during deployment.

Readiness probes tell Kubernetes when a pod is ready to receive traffic.

Liveness probes tell Kubernetes when a process should be restarted.

Resource requests help the scheduler place workloads.

Resource limits reduce the risk that one workload overwhelms the node.

Rolling updates reduce the chance that a deployment takes the service offline.

The API service is designed as a user-facing workload. It is routable, replicated, health-checked, and updateable.

The worker service has a different contract.

It also runs as a Deployment, but it does not expose a public HTTP Service. Its role is to consume queue work and interact with Postgres. That distinction matters.

Not every platform component should be routable.

Some components represent internal execution capacity.

The LLM service has its own Deployment and Service. Even when the local lab uses a mock runtime, the deployment shape prepares the platform for a real model-serving backend later. It also reflects the fact that inference workloads often have different resource profiles than API or worker services.

From an architecture perspective, each Deployment should communicate what kind of workload it represents.

The API is request-facing.

The worker is queue-facing.

The LLM service is inference-facing.

The observability service is operator-facing.

That separation is the foundation of a manageable platform.

Routing with Ingress

The project defines `ai-platform.local` as the local platform entry point.

The Ingress routes distinct URL prefixes to distinct services:

Path	Destination
<code>/api</code>	API service
<code>/llm</code>	LLM service
<code>/observability</code>	Observability service
<code>/dashboard</code>	Dashboard service

For a local lab, this keeps access simple.

For a production platform, this is the beginning of an API gateway strategy.

Ingress is not just a networking detail. It defines the product boundary.

It answers:

What can users or applications reach?

Which services are exposed?

Which services remain internal?

Where should TLS, authentication, rate limiting, request size controls, and routing policies eventually live?

In AI platforms, routing design matters because different services have different risk profiles.

An API endpoint may receive user questions.

An LLM endpoint may expose expensive model-serving capacity.

An observability endpoint may expose traces, prompts, metadata, and operational data.

A dashboard may expose platform status or business-facing insights.

The routing layer should make those boundaries explicit.

Configuration and Secrets

The project separates configuration from code using ConfigMaps and Secrets.

This is an essential platform pattern.

Container images should not be rebuilt every time an environment value changes. The same image should be able to move across local, staging, and production environments while configuration changes externally.

The API and worker services build database and Redis URLs from environment variables.

The LLM service receives runtime settings such as:

```
LLM_RUNTIME
DEFAULT_MODEL
```

This keeps runtime behavior configurable without changing application code.

Secrets are handled separately from ConfigMaps because sensitive values require a different control surface.

The local Secret manifests demonstrate the shape of the dependency. They show how credentials are injected into workloads. But they are not a production-grade secret management solution.

In production, secrets should come from a managed secret store, External Secrets, sealed secrets, or a cloud-native equivalent.

That distinction matters.

A learning lab can use example Secrets to teach the interface.

A production AI platform needs stronger credential management, access controls, rotation policies, and auditability.

The architectural lesson is clear: configuration belongs outside code, and secrets require a secure lifecycle.

Data Services: Postgres and Redis

The Kubernetes stack includes Postgres and Redis because AI platforms are stateful in multiple ways.

Postgres represents durable state.

In this platform, Postgres can support application metadata, job records, document data, vector-backed patterns, and other persistent records. The Kubernetes deployment models Postgres as a workload with persistent storage through a PersistentVolumeClaim.

That makes the data tier visible as a first-class platform dependency.

Redis represents queueing and asynchronous coordination.

It allows API replicas and worker replicas to scale differently. The API tier can remain responsive while workers process background jobs independently.

This separation is especially important in AI systems because some work is user-facing and some work is long-running.

The API should not be blocked by background processing.

The worker tier should be able to scale based on processing volume.

Redis provides the coordination layer between those responsibilities.

The project also makes the local-to-production gap visible.

A single Postgres pod is not the same as a highly available database.

Redis durability requirements need to be documented.

Backups need to be planned.

Storage classes need to be selected intentionally.

The value of this phase is that those questions now have a defined place in the architecture.

Autoscaling and Resource Management

The project defines HorizontalPodAutoscalers for the API service, worker service, and LLM service.

The API service can scale from two to six replicas based on CPU and memory utilization.

The worker service can scale from one to eight replicas based on CPU.

The LLM service can scale from one to three replicas based on CPU.

This is a practical first version of autoscaling.

It teaches how Kubernetes connects resource signals to replica counts and why resource requests and limits matter.

Without resource requests, the scheduler cannot make informed placement decisions.

Without limits, runaway workloads can put neighboring services at risk.

For AI workloads, CPU and memory are only the starting point.

A production platform may need AI-specific scaling signals:

- queue depth for worker scaling
- request latency for API scaling
- token throughput for model-serving capacity
- GPU utilization for LLM services
- retrieval latency for RAG systems
- vector database saturation for search workloads

The local HPA manifests introduce the scaling control surface. A production version would connect that control surface to richer telemetry.

This is an important platform concept: autoscaling should eventually align with the actual bottleneck.

Scaling an API service based on CPU may help request throughput.

Scaling workers based on queue depth may better match background workload demand.

Scaling model services based on GPU utilization or token throughput may better match inference capacity.

The right scaling signal depends on the workload.

Network Policies as a Security Baseline

The project includes NetworkPolicy objects that introduce a safer network posture.

One policy establishes default-deny ingress behavior.

Another allows traffic from the platform namespace and the ingress-nginx namespace.

This is a valuable security baseline.

Kubernetes service discovery makes it easy for workloads to find each other. But easy discovery does not mean every pod should be reachable from every other pod.

Network policies help turn the cluster from an open network into a platform with intentional communication paths.

For an AI platform, this matters because the system contains sensitive and expensive surfaces:

- API endpoints may receive user data.
- Retrieval services may access private documents.
- Model services may expose expensive inference capacity.
- Observability services may contain prompts, responses, metadata, traces, and failure details.
- Databases and queues should not be reachable from unrelated workloads.

The local policy is intentionally simple, but it introduces the right habit:

Default deny first.

Then allow only the traffic the platform actually needs.

That habit becomes more important as the platform begins handling sensitive enterprise data, private documents, PHI, PII, or regulated workloads.

Observability as a Platform Requirement

Kubernetes can run the workloads, but observability tells the platform team whether those workloads are behaving well.

The project treats observability as part of the platform boundary. It routes traffic to an observability service and recognizes dashboards as part of the deployed stack.

That is the right model for AI systems.

Traditional metrics like CPU, memory, request latency, and error rate are necessary, but they are not enough.

AI platforms also need to understand:

Are retrieved sources relevant?

Are responses grounded in context?

How many tokens are requests consuming?

Is model latency increasing?

Are queue depths growing?

Are prompt or model versions changing behavior?

Are dependency failures isolated or cascading?

Are services healthy during rollout?

Are dashboards and traces available before incidents occur?

Observability should not be bolted on after deployment. It should be deployed with the platform.

The production checklist calls out centralized logs, request latency, error rate, token usage, retrieval latency, queue depth, trace propagation, service health dashboards, AI reliability dashboards, and alerts.

That is the right level of concern for operating AI workloads.

Local Deployment with kind

The project includes a local `kind` cluster configuration and deployment scripts.

This makes the Kubernetes platform testable without needing a cloud cluster.

The local workflow includes a script to create a `kind` cluster with ingress ports mapped and another script to apply the manifests.

The deployment guide also highlights an important operational detail: local images must be built and loaded into the `kind` cluster.

The manifests reference local image names such as:

```
ai-platform/api-service:local  
ai-platform/worker-service:local  
ai-platform/llm-service:local  
ai-platform/observability-service:local
```

This is a useful learning point.

Kubernetes does not deploy source code.

It deploys container images.

That boundary matters because API images, worker images, model-serving images, and observability images can have different dependencies, build times, release cycles, and resource profiles.

A local `kind` setup makes that deployment boundary concrete.

It also gives the platform a safe environment for testing manifests, ingress, services, health checks, image loading, rollouts, and basic scaling behavior.

Rolling Deployments and Rollbacks

The deployment guide includes commands for updating a Deployment image, watching rollout status, and rolling back.

That may seem like a small operational detail, but it is architecturally important.

AI systems change often.

Teams update API code, worker logic, prompt templates, retrieval code, model versions, embedding models, ranking logic, dashboard definitions, and observability instrumentation.

Each change needs a deployment path and a rollback path.

Kubernetes Deployments provide the basic mechanics, but platform teams still need release discipline.

A strong deployment process should:

- version images immutably

- use readiness checks before routing traffic
- monitor latency, error rate, token usage, and platform health during rollout
- track model and prompt versions
- validate retrieval and inference behavior before broad release
- roll back quickly when behavior regresses

The project does not claim to solve all release governance. It creates the deployment foundation for it.

That foundation matters because AI regressions may not look like traditional software failures.

A pod may be healthy, but answer quality may drop.

The API may return `200 OK`, but token usage may spike.

A rollout may succeed technically while model behavior worsens.

This is why deployment mechanics and AI observability need to work together.

Production Readiness

The production-readiness checklist is one of the most useful parts of the project because it separates “runs locally” from “ready for production.”

A local Kubernetes deployment can validate the platform shape.

A production AI platform needs stronger guarantees.

The checklist covers several categories.

Deployment Safety

Production deployments need immutable image digests, readiness probes, rollback procedures, controlled database migrations, and clearly defined resource requests and limits.

These controls reduce the risk that a rollout breaks the platform.

Security

Production security requires real secret management, TLS, authentication, image scanning, non-root pods, and network restrictions.

This is especially important for AI systems because prompts, responses, traces, retrieved context, and source documents may contain sensitive information.

Reliability

Production reliability requires probes, autoscaling, backups, Redis durability decisions, storage choices, and PodDisruptionBudgets.

AI platforms depend on both stateless and stateful services. Reliability needs to cover both.

Observability

Production observability requires logs, metrics, traces, dashboards, and alerts.

For AI systems, that also includes AI-specific metrics such as token usage, retrieval latency, model latency, source grounding, queue depth, and quality signals.

AI-Specific Readiness

The AI-specific section is especially important.

Production readiness for AI includes:

- model runtime memory requirements
- GPU requirements
- concurrency limits
- RAG evaluation
- prompt versioning
- model version tracking
- embedding version tracking
- token budgets

This is the difference between a generic Kubernetes deployment and an AI platform deployment.

A generic deployment asks whether the pods are running.

An AI platform deployment asks whether the system is reliable, explainable, cost-aware, observable, and safe to operate under changing model and data behavior.

Local Choices and Production Replacements

The project is intentionally clear about the difference between local lab choices and production replacements.

That is one of its strengths.

The local version teaches the concept.

The production checklist shows what needs to change before real workloads depend on it.

Common replacements include:

Local Choice	Production Replacement
Example Kubernetes Secrets	Managed secrets, External Secrets, sealed secrets, or cloud-native secret management
Single Postgres pod	Managed Postgres or a high-availability database operator
Mock LLM service	GPU-backed model-serving runtime, vLLM, TGI, or managed inference endpoint
Basic nginx ingress	Managed ingress or load balancer with TLS
CPU/memory HPA	Scaling based on queue depth, latency, token throughput, or GPU utilization
Local dashboards	Managed Grafana, Phoenix, Prometheus, or equivalent observability stack

This is exactly how a learning lab should be structured.

It does not pretend the local setup is production-ready.

It shows the path from local deployment shape to production platform requirements.

Why This Matters for AI Data Platforms

AI data platforms sit at the intersection of application services, data systems, model-serving infrastructure, retrieval systems, orchestration, security, and observability.

A platform architecture needs to answer more than:

Can the model generate a response?

It needs to answer:

Can the system be deployed reliably?

Can services scale independently?

Can traffic be routed clearly?

Can secrets and configuration be managed safely?

Can stateful dependencies be identified and protected?

Can background processing scale separately from user-facing APIs?

Can model-serving capacity be isolated?

Can observability be accessed as part of the platform?

Can network traffic be restricted intentionally?

Can rollouts and rollbacks be performed safely?

Can production gaps be documented before real workloads depend on the system?

The Kubernetes deployment project creates a practical blueprint for those questions.

It organizes the AI stack around deployable, observable, and scalable boundaries.

That is the architectural move that matters.

Models may be the visible feature, but platforms make them dependable.

Architect's Takeaway

The value of this phase is not that it creates the final production platform.

The value is that it organizes the AI system around operational boundaries.

The earlier phases proved that individual services could exist. This phase asks whether those services can become a coherent platform.

The Kubernetes deployment validates several important platform ideas:

- services are independently deployable
- traffic has a defined ingress path
- configuration and secrets are externalized
- persistent and queueing services are modeled explicitly
- readiness and liveness probes are included
- resource requests and limits are defined
- rollout and rollback mechanics exist
- autoscaling is introduced
- network policy starts from a safer baseline
- observability is treated as part of the platform
- production gaps are documented instead of hidden

That is the mindset needed for real AI infrastructure.

Kubernetes is not a magic layer for AI. It does not automatically solve model quality, retrieval accuracy, cost control, data privacy, or production reliability.

What it does provide is a disciplined operating model for complex systems.

This project uses Kubernetes to bring together the major building blocks of an AI application stack: APIs, workers, model serving, stateful data, queues, observability, ingress, scaling, and security boundaries.

For anyone building AI systems beyond prototypes, that is the transition that matters.

The journey from notebook to production does not end with a model endpoint.

It ends with a platform that can be deployed, inspected, scaled, secured, rolled back, and improved.